

[Client Usage](#)[TanStack React Query \(★\)](#)[Setup](#)**Version: 11.x**

TanStack React Query

Compared to our [classic React Query Integration](#) this client is simpler and more TanStack Query-native, providing factories for common TanStack React Query interfaces like QueryKeys, QueryOptions, and MutationOptions. We think it's the future and recommend using this over the classic client, [read the announcement post](#) for more information about this change.

**TIP**

You can try this integration out on the homepage of tRPC.io: <https://trpc.io/?try=minimal-react#try-it-out>

▶ ? Do I have to use an integration?

Setup

1. Install dependencies

The following dependencies should be installed

[npm](#)[yarn](#)[pnpm](#)[bun](#)[deno](#)

```
npm install @trpc/server @trpc/client @trpc/tanstack-react-query @tans
```

2. Import your `AppRouter`

Import your `AppRouter` type into the client application. This type holds the shape of your

entire API.

utils/trpc.ts

```
import type { AppRouter } from '../server/router';
```



TIP

By using `import type` you ensure that the reference will be stripped at compile-time, meaning you don't inadvertently import server-side code into your client. For more information, [see the Typescript docs](#).

3a. Set up the tRPC context provider

In cases where you rely on React context, such as when using server-side rendering in full-stack frameworks like Next.js, it's important to create a new QueryClient for each request so that your users don't end up sharing the same cache, you can use the `createTRPCContext` to create a set of type-safe context providers and consumers from your `AppRouter` type signature.

utils/trpc.ts

```
import { createTRPCContext } from '@trpc/tanstack-react-query';
import type { AppRouter } from '../server/router';

export const { TRPCProvider, useTRPC, useTRPCClient } = createTRPCContext<AppRouter>();
```

Then, create a tRPC client, and wrap your application in the `TRPCProvider`, as below. You will also need to set up and connect React Query, which [they document in more depth](#).



TIP

If you already use React Query in your application, you **should** re-use the `QueryClient` and `QueryClientProvider` you already have. You can read more about the QueryClient initialization in the [React Query docs](#).

components/App.tsx

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { createTRPCClient, httpBatchLink } from '@trpc/client';
import { useState } from 'react';
import { TRPCProvider } from '../utils/trpc';

function makeQueryClient() {
  return new QueryClient({
    defaultOptions: {
      queries: {
        // With SSR, we usually want to set some default staleTime
        // above 0 to avoid refetching immediately on the client
        staleTime: 60 * 1000,
      },
    },
  });
}

let browserQueryClient: QueryClient | undefined = undefined;

function getQueryClient() {
  if (typeof window === 'undefined') {
    // Server: always make a new query client
    return makeQueryClient();
  } else {
    // Browser: make a new query client if we don't already have one
    // This is very important, so we don't re-make a new client if React
    // suspends during the initial render. This may not be needed if we
    // have a suspense boundary BELOW the creation of the query client
    if (!browserQueryClient) browserQueryClient = makeQueryClient();
    return browserQueryClient;
  }
}

export function App() {
  const queryClient = getQueryClient();
  const [trpcClient] = useState(() =>
    createTRPCClient<AppRouter>({
      links: [
        httpBatchLink({
          url: 'http://localhost:2022',
        }),
      ],
    }),
  );

  return (
    <QueryClientProvider client={queryClient}>
      <TRPCProvider trpcClient={trpcClient} queryClient={queryClient}>
        { /* Your app here */ }
      </TRPCProvider>
    </QueryClientProvider>
  );
}
```

3b. Set up with Query/Mutation Key Prefixing enabled

If you want to prefix all queries and mutations with a specific key, see [Query Key Prefixing](#) for setup and usage examples.

3c. Set up without React context

When building an SPA using only client-side rendering with something like Vite, you can create the `QueryClient` and tRPC client outside of React context as singletons.

utils/trpc.ts

```
import { QueryClient } from '@tanstack/react-query';
import { createTRPCClient, httpBatchLink } from '@trpc/client';
import { createTRPCOptionsProxy } from '@trpc/tanstack-react-query';
import type { AppRouter } from '../server/router';

export const queryClient = new QueryClient();

const trpcClient = createTRPCClient<AppRouter>({
  links: [httpBatchLink({ url: 'http://localhost:2022' })],
});

export const trpc = createTRPCOptionsProxy<AppRouter>({
  client: trpcClient,
  queryClient,
});
```

components/App.tsx

```
import { QueryClientProvider } from '@tanstack/react-query';
import React from 'react';
import { queryClient } from '../utils/trpc';

export function App() {
  return (
    <QueryClientProvider client={queryClient}>
      {/* Your app here */}
    </QueryClientProvider>
  );
}
```

4. Fetch data

You can now use the tRPC React Query integration to call queries and mutations on your API.

components/user-list.tsx

```
import { useMutation, useQuery } from '@tanstack/react-query';
import { useTRPC } from '../utils/trpc';

export default function UserList() {
  const trpc = useTRPC(); // use `import { trpc } from '../utils/trpc'` if you're using the si

  const userQuery = useQuery(trpc.getUser.queryOptions({ id: 'id_bilbo' }));
  const userCreator = useMutation(trpc.createUser.mutationOptions());

  return (
    <div>
      <p>{userQuery.data?.name}</p>

      <button onClick={() => userCreator.mutate({ name: 'Frodo' })}>
        Create Frodo
      </button>
    </div>
  );
}
```